

CityScan

המדריך המלא — ארכיטקטורה, קוד, והצגה

FastAPI + Gemini

MongoDB

TypeScript

Node.js + Express 5

React 19 + Vite

Leaflet

אפליקציית Web לדיווח על מפגעים עירוניים

~7,000 שורות קוד · 46 בדיקות · CI ירוק

אוגוסט 2026

תוכן עניינים

חלק א' — הפריקט

- מה זה, וההצגה של 90 שניות
- שלושה שירותים
- הכלים ולמה נבחרו
- הארכיטקטורה — שני צירים

חלק ב' — איך זה עובד

- צינור ה-Middleware
- הזרימה המלאה — מסלול של דיווח
- זיהוי כפילויות — שתי רמות
- שלוש השכבות שהכי מבלבלות

חלק ג' — מה יש בפועל

- Backend — כל תיקייה, כל קובץ
- Frontend — כל קובץ
- שירות ה-AI
- בדיקות

חלק ד' — המצב

- מצב דמו — איך זה עובד
- מה חי ומה לא
- סביבות עבודה ואיך להריץ
- שיקולי עלות

חלק ה' — הסיפור

- מה שונה מ-SmartCollect ולמה
- חמשת הבאגים שנמצאו ותוקנו
- איך להסביר את זה לאנשים
- מה נשאר

1. מה זה

אפליקציית Web לדיווח על מפגעים עירוניים. תושב לוחץ על מפה, בוחר סוג מפגע — בור, פנס שבור, פסולת, הצפה — מצלם ומדווח. הדיווחים מוצגים לכולם על מפה משותפת, ולעירייה יש פאנל ניהול לעדכון סטטוס.

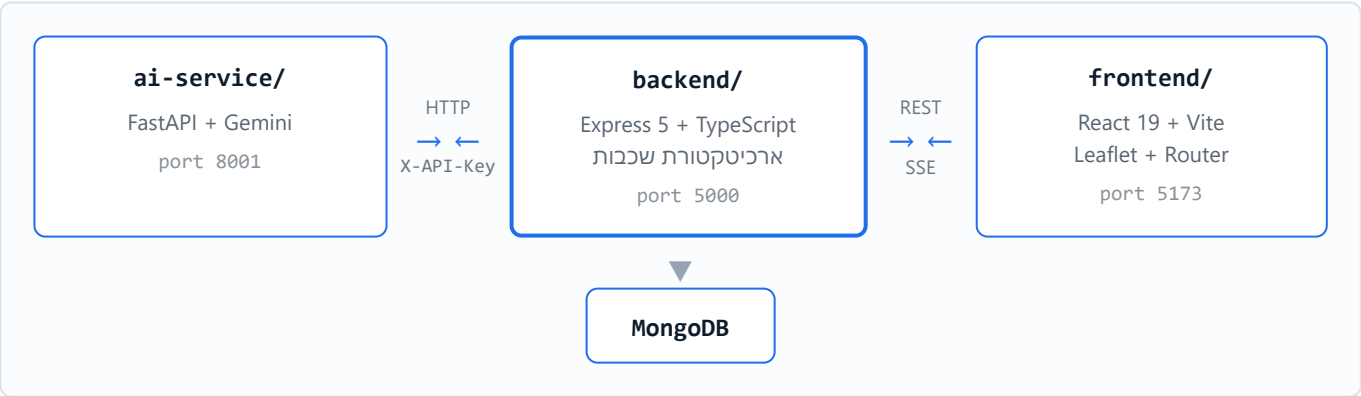
ההצגה של 90 שניות

- 1. מה זה — "מערכת דיווח מפגעים עירוניים. תושב לוחץ על מפה, מצלם, מדווח."
- 2. הארכיטקטורה — "שלושה שירותים: API ב-React/TypeScript ב-Node בארכיטקטורת שכבות קפדנית, ושירות AI נפרד בפייתון."
- 3. מה מעניין טכני — "מניעת כפילויות בשתי רמות: שאילתה גאוגרפית תופסת את הוודאי במילישניות, ו-LLM מכריע רק במעורפל."
- 4. מה למדתי — הבאגים. זה החלק שמבדיל אותך משאר המועמדים.

הבעיה שזה פותר

עיריות טובעות בדיווחים כפולים — אותו בור מדווח אחת עשרה פעמים. ותושב לא יודע אם מישהו כבר דיווח. CityScan עונה על שני הצדדים.

2. שלושה שירותים



שירות	גודל	מה בפנים
backend/	74 קבצים ~3,900 שורות	11 Node.js, Express 5, TypeScript (ESM), Mongoose. תיקיות בארכיטקטורת שכבות
frontend/	13 קבצים ~2,900 שורות	React 19, Vite 7, react-leaflet, Bootstrap. חמישה מסכים
ai-service/	237 שורות	FastAPI + Google Gemini. שני נתיבים מוגנים ב-X-API-Key

למה שירות AI נפרד ולא בתוך ה-Backend

שלוש סיבות שכדאי לומר: המערכת האקולוגית של ML היא פייתון; זה מבודד את התלות — אם ה-AI נופל, המערכת ממשיכה לעבוד; וזה מאפשר להחליף ספק LLM בלי לגעת בלוגיקה העסקית.

3. הכלים ולמה נבחרו

כלי	למה הוא ולא אחר
TypeScript	מופעל עם <code>strict</code> , <code>exactOptionalPropertyTypes</code> , <code>noUncheckedIndexedAccess</code> — ההגדרות הקפדניות, לא ברירת המחדל
Express 5	מינימלי. השכבות נבנו ידנית ולכן הן מפורשות ולא מוסתרות ע"י framework
MongoDB + Mongoose	השאלתה המרכזית גאוגרפית. אינדקס <code>2dsphere</code> נותן <code>\$geoWithin</code> נייטיב. Atlas M0 חינמי ולא נכבה
Zod	<code>z.infer</code> גוזר את הטיפוס מהסכמה — הוולידציה והטיפוס לא יכולים להתפצל
JWT + bcrypt	Stateless. סיסמאות מגובבות ב-10 סבבים
sharp	הספרייה המהירה ביותר ב-Node. WebP + הסרת EXIF
SSE	הזרימה חד-כיוונית. קל בהרבה מ-socket.io, עובר דרך כל פרוקסי, מתחבר מחדש לבד
Vitest + mongodb-memory-server	DB אמיתי חד-פעמי לכל ריצה, בלי לדרוש Docker
Leaflet + OpenStreetMap	0\$, בלי מפתח API ובלי כרטיס אשראי
Nominatim	גיאוקודינג דו-כיווני חינמי. אותה מחסנית כמו SmartCollect

"למה לא Google Maps?"

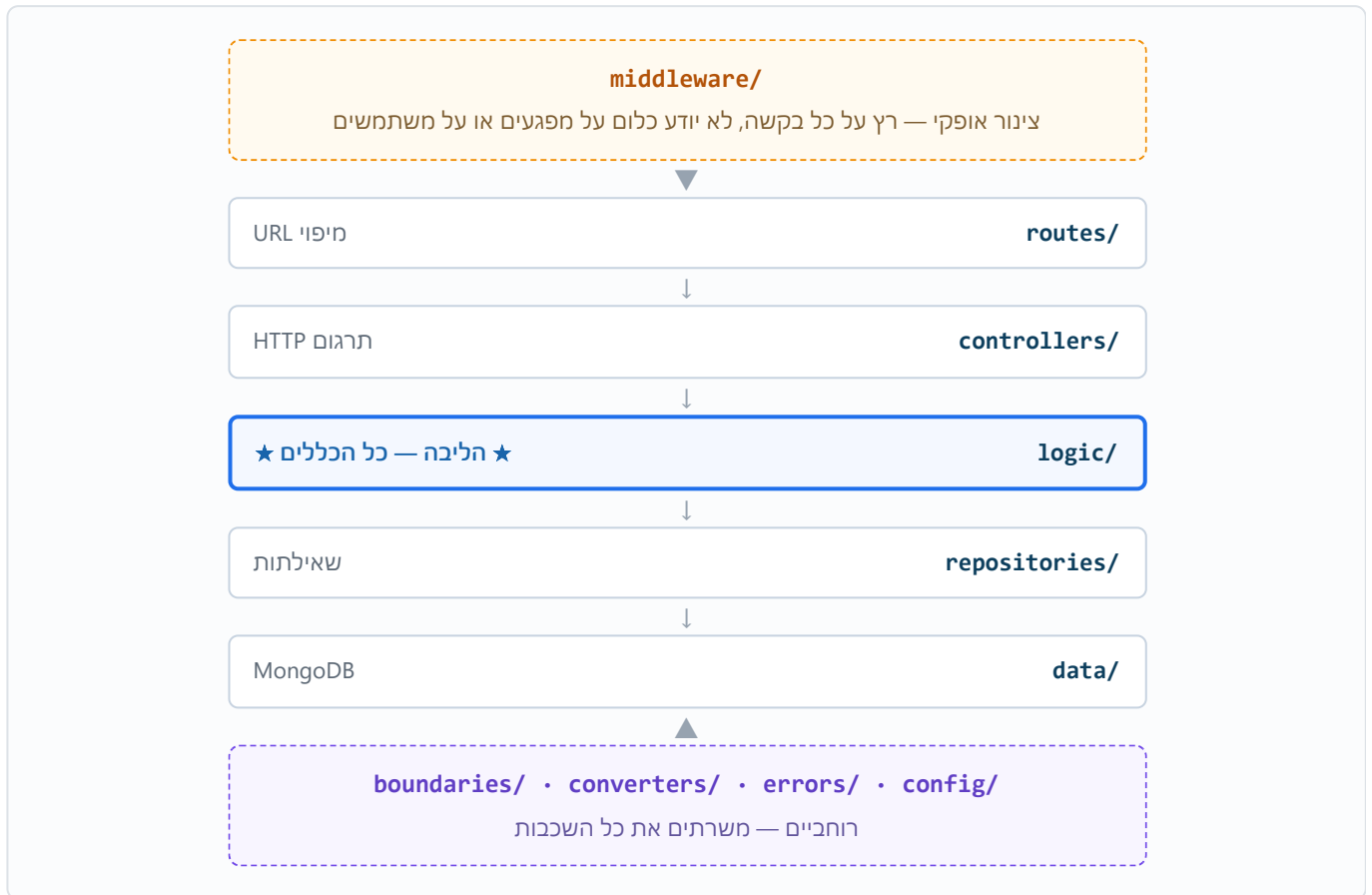
"בדקתי. מאז מרץ 2025 גוגל ביטלה את הקרדיט של \$200 — Dynamic Maps מקבל 10,000 טעינות בחודש ואז כ-\$7 לכל 1,000, וחובה כרטיס אשראי. אפשר להגביל עם Quota יומי, שזו תקרה קשיחה אמיתית — לא Budget Alerts, שהם רק התראה. אבל כשהמכסה נגמרת המפה מתה עד חצות. לפרויקט בעלות אפס זה לא השתלם."

"למה MongoDB ולא PostgreSQL?"

"השאלתה המרכזית שלי גאוגרפית, ו-Mongo נותן אותה נייטיב. PostGIS חזק יותר, אבל בסביבה עם זיכרון מוגבל אין מקום ל-Postgres מקומי, והחלופות המנוהלות החינמיות נרדמות. Atlas M0 לא נרדם. **והנקודה החשובה** — הסטנדרט ההנדסי הוא ההפרדה לשכבות, לא מנוע האחסון. בפרויקט הקודם שלי החלפתי MongoDB ב-PostgreSQL באמצע, ורק שתי שכבות השתנו."

4. הארכיטקטורה — שני צירים

זו הנקודה שהכי חשוב להעביר נכון. יש **מחסנית אנכית** של שכבות, ויש **צינור אופקי** שרץ לפניה. הם לא אותו דבר.



חוק הזהב: חץ אחד למטה

data/ לא יודע ש- **logic/** קיים. **logic/** לא יודע ש-HTTP קיים. הקונטרולים מכירים רק **ממשקים**, לא מימושים. התוצאה המעשית: אפשר לבדוק את הלוגיקה העסקית בלי להרים שרת, ואפשר להחליף את ספק ה-LLM בלי לגעת בה.

המקבילה ב-Spring Boot

המבנה מועתק מפרויקט SmartCollect, כולל מוסכמות השמות — ואפילו האי-סימטריה שבהן: שירותים ברבים (**HazardsService**), קונטרולים וישויות ביחיד (**HazardEntity** , **HazardController**).

SmartCollect	CityScan
DTOs — boundaries/	boundaries/
controllers/	controllers/
converters/	converters/
@Entity — data/	data/
@ResponseStatus — errors/	errors/
זוה	logic/impl/ + logic/
Spring Data — repositories/	repositories/
אין — הכל ב- application.properties	config/
אין — ספרינג מספק מובנה	middleware/
אין — ספרינג מזריק אוטומטית	container.ts

חמשק מול מימוש

לכל שירות יש שני קבצים, וזה מבלבל בהתחלה:

קובץ	מה יש בו
logic/ai.service.ts	מה אפשר לעשות — רשימת פונקציות בלי גוף
logic/impl/gemini.ai.service.impl.ts	איך — הקוד שבאמת רץ

מכיר רק את הראשון. לכן החלפת ספק ה-LLM היא קובץ חדש ושורה אחת ב- hazards.service.impl.ts
container.ts.

5. צינור ה-Middleware

ב-Spring Boot רוב הדברים האלה הם אנוטציות שה-framework מספק. ב-Express הם קוד מפורש — וזה יתרון הסברתי: רואים בדיוק מה קורה.

ב-CityScan (קוד)	ב-Spring Boot (אנוטציה)
errorHandler.ts	@ControllerAdvice
requireRole.ts	@PreAuthorize
validate.ts	jakarta.validation + @Valid
auth.middleware.ts	Spring Security Filter Chain
rateLimiter.ts	— אין מקבילה מובנית

למה זה לא "שכבה"

שכבה עוסקת בדומיין — מפגעים, משתמשים. Middleware הוא חוצה-דומיין: אותו קוד אימות רץ על מפגעים, על משתמשים ועל לוגים, ולא יודע במה הוא מטפל. לכן הוא ציר אחר.

6. הזרימה המלאה

1 **cors** מקור מותר? בפרודקשן חובה FRONTEND_URL

2 **helmet** כותרות אבטחה

3 **express.json** פענוח גוף, מקסימום 8mb

4 **rateLimiter** לא חרגת ממכסה? (נתיב ה-SSE פטור)

5 **optionalAuth** יש טוקן? → req.user

6 **demoRestrict** אדמין-דמו? → חסום כתיבה

— עד כאן גלובלי על כל בקשה. מכאן לפי נתיב —

7 **authMiddleware** חייב טוקן, אחרת 401

8 **validate(schema)** נכשל → **BadRequestException**

== נכנסים לשכבות ==

9 **HazardController.create** 3 שורות: קורא, מאציל, שולח

10 ★ **HazardsServiceImpl.create** ★ כל הכללים

← repository.findNearbyUnresolved() — \$geoWithin, רדיוס 50 מ'

← אותו סוג ברדיוס? זורק **ConflictException** (409)

← סוג אחר? ai.checkDuplicate() — נופל בחן אם השירות מת

← photos.saveMany() — WebP, ניקוי EXIF, בלבד

← repository.create() → data/ → MongoDB

← events.emit() — SSE לכל הדפדפנים המחוברים

11 **converter.toBoundary()** entity נקי → JSON

12 **res.status(201).json()** התשובה ללקוח

וכשמשהו נכשל

זורק **ConflictException**

HazardsServiceImpl

↓ תופס **asyncHandler**

קופצים לסוף הצינור

errorHandler

{ message, status: 409,
code: "DUPLICATE_HAZARD",
timestamp, path }

אף מקום בפרויקט חוץ מ- `errorHandler.ts` לא כותב תשובת שגיאה. השירות זורק חריגה טיפוסית שנושאת את קוד ה-HTTP שלה; הוא לא בוחר סטטוס ולא בונה JSON.

7. זיהוי כפילויות — שתי רמות

זה הפיצ'ר שהכי שווה להסביר לעומק.

רמה 1 — דטרמיניסטית

מפגע לא פתור מאותו סוג בתוך 50 מטר הוא כפילות, נקודה. שאילתת `$geoWithin` עם `$centerSphere` על אינדקס `2dsphere`. בלי AI, בלי השהיה, עובד גם כשהכל למטה.

```
location: { $geoWithin: { $centerSphere: [[lng, lat], radiusMeters / 6378100] } }
```

מקבל רדיוס ברדיאנים, ולכן החלוקה ברדיוס כדור הארץ במטרים. החישוב הזה חי ב- `$centerSphere` — השירות מבקש "מפגעים ברדיוס 50 מטר" ולא יודע מה זה רדיאן. `repositories/`

רמה 2 — שיפוט

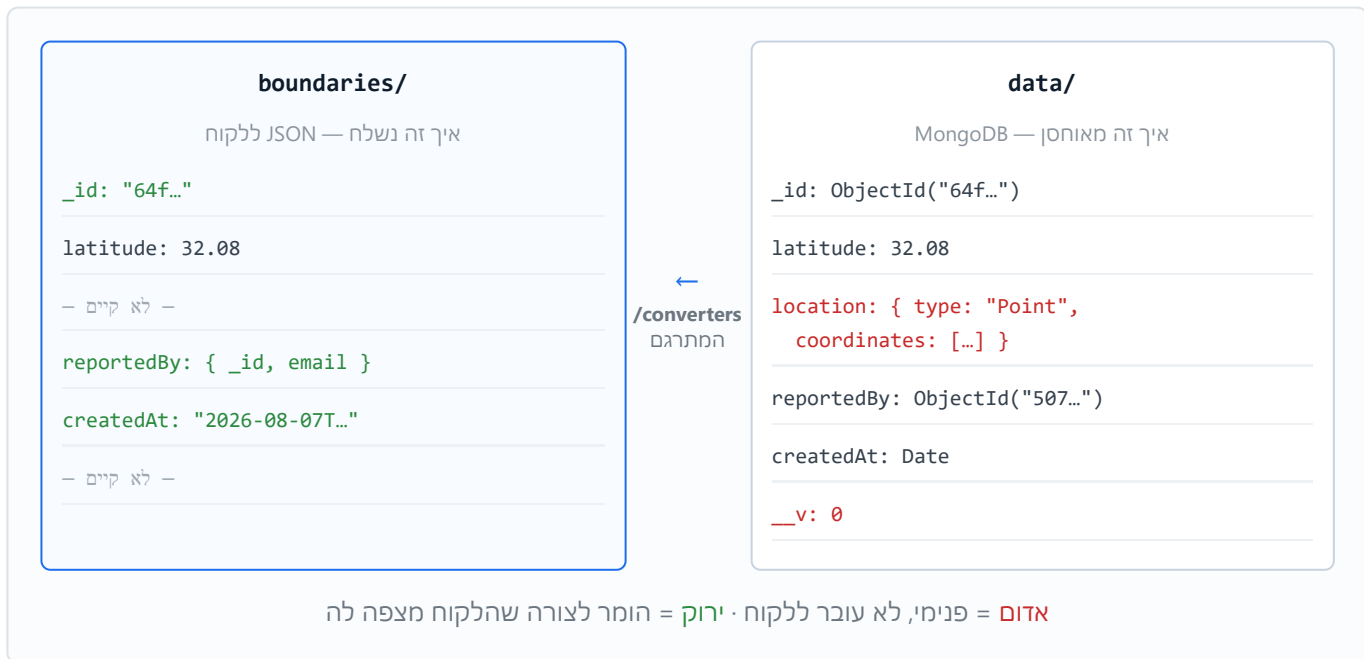
רצה רק כשיש בסביבה מפגעים מסוג אחר, שם השאלה באמת מעורפלת: "פסולת" ו"הצפה" במרחק חמישה מטרים עשויים להיות אותה בעיה פיזית. השאלה הולכת ל-Gemini.

Fail-open — החלטת תכנון מודעת

אם שירות ה-AI לא זמין, הדיווח עובר. תושב חייב תמיד להיות מסוגל לדווח על מפגע, גם כשתלות אופציונלית נופלת. זו גם הסיבה שבדיקת הכפילויות לא הועברה לכלי אוטומציה חיצוני: היא על הנתיב הקריטי, המשתמש ממתין לתשובה, וזו לוגיקה עסקית ליבתית שצריכה להיות ב-git ומכוסה בבדיקות.

8. שלוש השכבות שהכי מבלבלות

אותו מפגע, בשלוש צורות שונות:



data/ — הצורה שנשמרת

סכמות Mongoose. השדה `location` קיים רק בשביל האינדקס `2dsphere` — הוא כפילות של `latitude` / `longitude`, ו-hook שומר אותו מסונכרן. הלקוח לא רואה אותו לעולם.

המלכודת הקלאסית ב-GeoJSON

הסדר הוא `[longitude, latitude]` — הפוך מהאינטואיציה ומאיך שכותבים קואורדינטות בדיבור.

boundaries/ — החוזה כלפי חוץ

שני דברים באותו קובץ: טיפוס התשובה, וסכמת Zod שבודקת את הבקשה.

```
export const createHazardSchema = z.object({
  type: z.enum(HAZARD_TYPES),
  latitude, // למספר, בין 90- ל-90
  longitude,
  hazardPhotos: z.array(photoPayload).max(10).optional(),
});

export type CreateHazardBoundary = z.infer<typeof createHazardSchema>;
```

השורה האחרונה היא הרעיון: **הטיפוס נגזר מהסכמה**, ולכן אי אפשר שהוולידציה תבדוק דבר אחד וה-TypeScript יאמין לדבר אחר.

converters/ — המתרגם

הקובץ הכי חשוב ביטחונית הוא `user.converter.ts`, כי הוא בונה את התשובה שדה-בשדה — ול-`UserBoundary` בכלל אין שדה `password`, אז הסיסמה פיזית לא יכולה לדלוף.

הקוד הישן (3 מופעים)	הקוד החדש
<pre>const o = user.toObject(); delete o.password;</pre> דולף כברירת מחדל — שדה חדש בסכמה נשלח ללקוח עד שמישהו יזכור למחוק	<pre>toBoundary(entity) { return { _id, email, role, ... } }</pre> חוסם כברירת מחדל — שדה חדש נשלח רק אם כתבת אותו במפורש

9. Backend — כל תיקייה, כל קובץ

backend/src/ — 74 קבצים, ~3,900 שורות

config/ — תצורה וסביבות

קובץ	שורות	תפקיד
env.ts	222	מקור האמת היחיד לתצורה. מאומת ב-Zod, מוקפא. מסרב לעלות בפרודקשן בלי סוד
loadEnv.ts	71	חייב להיות הייבוא הראשון. טוען {פרופיל}.env ואז .env. ערך ריק = "לא סופק"
db.ts	33	חיבור Mongoose. זורק שגיאה, לא process.exit

כלל שנאכף בכל הפרויקט

אף מודול לא קורא process.env בזמן ייבוא. כולם מייבאים config. זה מה שמונע את חזרת באג טעינת הסביבה.

data/ — ישויות ו-enums

קובץ	שורות	תפקיד
demoFixtures.ts	180	15 מפגעי הדמו בתל אביב
hazard.entity.ts	76	סכמת המפגע. longitude / latitude שטוחים וגם location ב-GeoJSON עם אינדקס 2dsphere
user.entity.ts	35	משתמש. password עם select: false
log.entity.ts	33	טבלת אודיט
enums.ts	26	סוגים, סטטוסים, תפקידים — מוגדרים פעם אחת

boundaries/ — חוזה ה-JSON

כל קובץ מייצא גם סכמת Zod וגם את הטיפוס, כש-z.infer גוזר את השני מהראשון.

קובץ	שורות	תפקיד
<code>hazard.boundary.ts</code>	144	יצירה, עדכון, שאילתות, בדיקת כפילות
<code>common.boundary.ts</code>	40	מזהה <code>ObjectId</code> , מבנה השיאה
<code>user.boundary.ts</code>	40	ללא שדה סיסמה — בכוונה
<code>auth.boundary.ts</code>	38	הרשמה, התחברות, דמו
<code>log.boundary.ts</code>	29	

`repositories/` — שאילתות בלבד

קובץ	שורות	תפקיד
<code>hazard.repository.ts</code>	150	הגאוגרפיה חיה כאן. <code>findNearbyUnresolved(lng, lat, מטרים)</code>
<code>user.repository.ts</code>	37	<code>findByEmailWithPassword</code> — הנתיב היחיד שטוען סיסמה
<code>log.repository.ts</code>	16	

`logic/` — ממשקים

קובץ	שורות	תפקיד
<code>hazards.service.ts</code>	51	כל מה שאפשר לעשות עם מפגע
<code>ai.service.ts</code>	47	הממשק שמאפשר להחליף Gemini ב-Groq
<code>events.service.ts</code>	37	פרסום אירועים ל-SSE
<code>demo.service.ts</code>	29	איפוס לתצורת המקור
<code>photoStorage.service.ts</code>	25	היום דיסק, מחר S3
<code>auth.service.ts</code>	24	
<code>users.service.ts</code> <code>logs.service.ts</code>	8 · 9	

קובץ	שורות	תפקיד
hazards.service.impl.ts	292	הקובץ החשוב ביותר. כפילויות, תזמור AI, תמונות, הרשאות בעלות
gemini.ai.service.impl.ts	122	לקוח ל-ai-service
localDisk.photoStorage.impl.ts	119	sharp → WebP, ניקוי EXIF
auth.service.impl.ts	109	JWT, bcrypt, חשבון האורח
demo.service.impl.ts	106	מחיקה מלאה וזריעה מחדש
emitter.events.service.impl.ts	37	EventEmitter פנימי
· users.service.impl.ts logs.service.impl.ts	28 · 38	

middleware/ · errors/ · converters/

קובץ	שורות	תפקיד
converters/hazard.converter.ts	83	משטח populate, בונה URLs, מסתיר location ו-__v
converters/user.converter.ts	41	בונה שדה-בשדה. הסיסמה לא יכולה לדלוף
6 + errors/app.exception.ts	6×8 + 35	503 / 409 / 404 / 403 / 401 / 400
middleware/errorHandler.ts	145	המקום היחיד שכותב שגיאה
middleware/validate.ts	82	Zod על כל נתיב
middleware/auth.middleware.ts	78	אימות חובה + אופציונלי
middleware/demoRestrict.ts	50	אדמין-דמו לקריאה בלבד
middleware/requireRole.ts	36	שומר תפקידים
· middleware/rateLimiter.ts asyncHandler.ts	20 · 32	

קובץ	שורות	תפקיד
controllers/hazard.controller.ts	135	כולל נקודת ה-SSE
controllers/demo.controller.ts	46	איפוס, עם השוואת סוד בזמן קבוע
controllers/auth · log · user	19 · 20 · 27	
routes/hazard.routes.ts	72	מדיניות ההרשאות קריאה מכאן לבד
routes/health · demo · auth · log · user	17 · 32 · 35 15 · 15 ·	
server.ts	48	נקודת הכניסה. סדר הייבוא קריטי
container.ts	96	כל גרף התלויות. ההזרקה של ספרינג, ידנית
app.ts	102	createApp() · בלי listen , בלי dotenv
scripts/seedDemo.ts clearReports.ts	48 · 55	

10. Frontend — כל קובץ

frontend/src/ — 13 קבצים, ~2,900 שורות

קובץ	שורות	תפקיד
components/ReportSidebar.tsx	602	טופס הדיווח — הקובץ הגדול ביותר
pages/AdminPage.tsx	541	פאנל הניהול, פילטרים, שינוי סטטוס
components/MapComponent.tsx	398	המפה, הסמנים, לחיצות
api/client.ts	398	קונפיגורציה, HTTP, טוקן, וכל קריאות ה-API
components/NavBar.tsx	293	ניווט + חיפוש כתובות (Nominatim)
pages/LoginPage.tsx	131	התחברות + כפתורי הדמו
pages/MyReportsPage.tsx	118	הדיווחים שלי
contexts/AuthContext.tsx	104	מצב אימות גלובלי
data/demoFixtures.ts	98	15 המפגעים לדמו בדפדפן
pages/RegisterPage.tsx	83	
App.tsx · main.tsx	67 · 69	Router, AuthProvider, טיפול ב-GPS
components/DemoBanner.tsx	27	הפס הצהוב

אי-עקביות מודעת שכדאי לדעת עליה

pages/ תואם ל-SmartCollect, אבל api/client.ts מחזיק לבדו את מה ששם מפוצל ל- core/ config, http,) session) ול- services/ (לפי דומיין).

הפיצול מתוכנן אך לא בוצע — הוא תוזמן יחד עם מחיקת מנגנון הדמו, וזו נדחתה עד שיהיה Backend פרוס. אם ישאלו על זה בראיון, זו התשובה.

11. שירות ה-AI

ai-service/app/main.py — 237 שורות. שני נתיבים מוגנים ב- X-API-Key :

נתיב	תפקיד
POST /check-duplicate	מקבל מפגעים קיימים + דיווח חדש, מחזיר is_duplicate
POST /analyze	מקבל תמונה ב-base64, מחזיר תיאור
GET /health	בדיקת חיים

המודל מוגדר ב- GEMINI_MODEL — כרגע gemini-3.5-flash, מקובע בכוונה ולא alias נייד שישתנה תחתיו בלי deploy.

12. בדיקות

46 בדיקות אינטגרציה מול mongod אמיתי שעולה נקי בכל ריצה — בלי צורך ב-Docker.

קובץ	שורות	מכסה
hazards.test.ts	301	חוזה ה-JSON, כפילויות, תמונות, הרשאות, שגיאות
auth.test.ts	150	הרשמה, התחברות, רגרסיית JWT, הגבלת הדמו
demoReset.test.ts	150	האיפוס והשומרים שלו
fixtureSync.test.ts	97	ששתי רשימות ה-fixtures לא נפרדו
sse.test.ts	95	זרם אירועים חי

שתי בדיקות ששוות להראות

מניעת מיפוי חשבונות: מייל לא קיים וסיסמה שגויה מחזירים *בדיוק אותה הודעה*, אחרת נקודת הקצה הופכת לכלי שמגלה אילו כתובות רשומות במערכת.

רגרסיה לבאג: בדיקה שמאמתת שהטוקן לא מתאמת מול הסוד הישן — כדי שהבאג שתוקן לא יחזור בשקט.

13. מצב דמו — איך זה עובד

יש שני מנגנונים נפרדים, וחשוב לא לבלבל ביניהם:

א. כספת localStorage	ב. חשבון אורח
מופעל ע"י	POST /auth/demo-login
דורש Backend	כן
הטוקן	מזויף — "demo-token"
איפה רץ	האתר ב-Vercel
	מקומית

באתר החי, 15 המפגעים נזרעים לכספת בטעינה ראשונה, וחסידת הכפילויות ברמה 1 מחושבת בדפדפן עם haversine.

למה ה-AI לא עובד בדמו — וזה בכוונה

ניתוח תמונות והכרעת רמה 2 דורשים מפתח Gemini. כל משתנה שמתחיל ב-VITE_ מוטמע לתוך קובץ ה-JavaScript ונקרא ע"י כל אחד שפותח את הבאנדל.

לכן שני הפיצירים האלה מושבתים בדמו ומחזירים הודעה מפורשת — עדיף להגיד "דורש שרת" מאשר לעבוד בזול על חשבון חשיפת המפתח.

14. מה חי ומה לא

רכיב	מצב
Vercel ב-Frontend	חי — city-scan-tawny.vercel.app, במצב דמו
Backend	לא פרוס. רץ רק מקומית
ai-service	לא פרוס. רץ רק מקומית
MongoDB Atlas	קלסטר קיים, ריק
CI ב-GitHub Actions	ירוק — 46 + build בדיקות בכל push
איפוס דמו לילי	קוד מוכן, ממתין לפריסת Backend

15. סביבות עבודה ואיך להריץ

`NODE_ENV` בוחר את הפרופיל, בסגנון `application-{profile}.properties` של ספרינג. סדר הטעינה: משתני מערכת אמיתיים ← `{פרופיל}.env` ← `.env`. הראשון שמגדיר מפתח מנצח.

test	production	development	
בזיכרון, נמחק	Atlas	Docker מקומי	מסד נתונים
נקבע בבדיקות	חובה ≤ 32 תווים	אופציונלי, מזהיר	JWT_SECRET
—	חובה	הכל מותר	CORS
כולל stack	הודעה גנרית	כולל stack	גוף שגיאה
מכובה	רשת Docker פנימית	localhost:8001	AI

הרצה

```
# מקומי MongoDB – פעם אחת אחרי אתחול מחשב
cd backend && npm run db:up

# נתונים למפה
npm run seed:demo

# שלושת השירותים יחד (מהשוש)
npm run dev
```

Sign in as Admin (Demo) ← `http://localhost:5173`

להדגמת חסימת כפילויות: לחץ על המפה ברחוב דיזנגוף ליד סמן קיים ודווח על **בור** נוסף. שני הבורות שם נמצאים 24 מטר זה מזה — בתוך הרדיוס.

```
npm test --prefix backend      # בדיקות 46
npm run build --prefix backend  # קומפילציה
```

16. שיקולי עלות

רכיב	עלות	הבחירה
מסד נתונים	0\$	MongoDB Atlas M0 — חינמי ולא נרדם
מפה	0\$	Leaflet + OSM, בלי מפתח ובלי כרטיס אשראי
גיאוקודינג	0\$	Nominatim
אחסון תמונות	0\$	דיסק. דחיסה הופכת את זה לבר-קיימא
LLM	0\$	מסלול חינמי של Gemini
Frontend	0\$	Vercel
CI	0\$	GitHub Actions, ריפו ציבורי
שרת ל-Backend	0\$	Azure B1s — 750 שעות חודשיות חינם ל-12 חודשים

החישוב שמאחורי דחיסת התמונות

תמונה מהטלפון היא ~1.5MB ב-base64. אחסון בתוך מסמך MongoDB היה מתקרב לתקרת 16MB למסמך, וממלא Atlas M0 (512MB) אחרי כ-100 דיווחים בלבד. אחרי `sharp` — שינוי גודל ל-1600px והמרה ל-WebP — התמונה יורדת ל-~150KB.

ניקוי ה-EXIF הוא לא רק חיסכון: תמונות מטלפון נושאות קואורדינטות GPS מוטמעות, והן מוצגות בפומבי על מפה. הדיווח כבר נושא מיקום מפורש — אין סיבה לפרסם גם את המקום המדויק שבו המדווח עמד.

17. מה שונה מ-SmartCollect ולמה

המבנה מועתק אחד-לאחד. מה שלא הועתק הוא החולשות — והמדריך של SmartCollect מפרט אותן בעצמו.

חולשה ב-SmartCollect	ב-CityScan
סיסמאות בטקסט גלוי	10, bcrypt סבבים
סיסמה כפרמטר ב-URL ("נכנסים ללוגים")	JWT ב- <code>Authorization: Bearer</code>
קריאות <code>require...</code> ידניות בכל מתודה	<code>requireRole</code> כ-middleware על הנתיב
אין <code>@ControllerAdvice</code> , וגוף השגיאה לא מוגדר — ההודעות לא מגיעות ללקוח	handler מרכזי עם חוזה מפורש
מעבר בין סביבות ע"י הסרת הערות מקוד	קבצי פרופיל אמיתיים
<code>compose.yaml</code> בלי <code>volume</code> — <code>down</code> מוחק את ה-DB	<code>image</code> , <code>named volume</code> מקובע, <code>healthcheck</code>
ולידציה ידנית בתוך כל <code>ServiceImpl</code>	Zod על הנתיב, סכמה אחת לטיפוס ולבדיקה

Docker — הבדל תפקידי

ב-SmartCollect אין `Dockerfile` בכלל: Docker היה רק נוחות פיתוח שהחזיקה את ה-PostgreSQL, וספרינג הרים את הקונטיינר לבד — ולכן חובה היה להפעיל Docker Desktop לפני ההרצה. ב-CityScan Docker אופציונלי בפיתוח, ובפריסה הוא הופך לסביבת הייצור עצמה.

18. חמשת הבאגים שנמצאו ותוקנו

זה החומר החזק ביותר לראיון, כי הוא מדגים אבחון ולא רק כתיבה.

1 — משתני הסביבה לא נטענו כלל

התסמין

ה-JWT תמיד נחתם עם סוד ברירת מחדל, ו-`AI_SERVICE_URL` תמיד `undefined` — כלומר כל מנגנון ה-AI היה מושבת בשקט.

הסיבה

`app.ts` קרא ל-`dotenv.config()` בשורה 12. אבל ב-ESM כל ה-`import` מתבצעים לפני גוף המודול, ו-`services/jwt.ts` קרא את `process.env` ברמת המודול — לפני ש-`dotenv` רץ.

2 — ה-Error Middleware היה קוד מת

הסיבה

כל controller עטף את עצמו ב- `try/catch` והחזיר `res.status(500)` בעצמו, ולכן `next(err)` לא נקרא אף פעם.

3 — הוולידציה הייתה פונקציה ריקה

הסיבה

היה `next()` ב- `middleware/validate.ts` ותו לא. הוא יוצא ומיובא, כך שנראה כאילו יש ולידציה — בעוד היא בפועל `if`-ים ידניים בתוך הקונטרולרים.

4 — הגבלת הדמו מעולם לא נתפסה

הסיבה

`demoRestrict` השווה מול `admin-demo@cityscan.demo` בעוד `auth.ts` יצר `guest_admin@cityscan.com`. שתי כתובות שונות — התנאי תמיד יצא "לא דמו".

5 — שירות ה-AI מעולם לא עבד

שתי תקלות עצמאיות, כל אחת הספיקה לבדה

- א. `ExistingHazard` הצהיר על `_id: str`. Pydantic מתייחס לקו תחתון מוביל כמאפיין פרטי ומוציא אותו מהמודל — השדה אף פעם לא אוכלס, וכל גישה זרקה. `/check-duplicate` החזיר **500 בכל קריאה**.
- ב. `gemini-1.5-flash` הוצא משימוש ומחזיר 404.

למה זה היה בלתי נראה

ה-Backend נופל בחן — **שירות מקולקל נראה בדיוק כמו "לא כפילות" בטוחה**. זה גם לקח על תכנון: fail-open הוא נכון, אבל הוא חייב לוג ברמת אזהרה, אחרת כשל שקט חי חודשים. ובחירת המודל החלופי דרשה בדיקה אמפירית: `list_models()` מפרסם את `gemini-2.5-flash`, אבל קריאה אליו מחזירה "no longer available to new users".

19. איך להסביר את זה לאנשים

שאלות שישאלו, ותשובות

"למה כל כך הרבה תיקיות לפרויקט קטן?"

"כי הקונטרולר היה 500 שורות שהחזיקו שאילתות, מתמטיקה גאוגרפית, קריאות AI ובדיקות הרשאה. אי אפשר היה לבדוק שום דבר מזה בלי להרים שרת. עכשיו הלוגיקה העסקית נבדקת בלי HTTP בכלל. המבחן שאני מציע: להחליף את ספק ה-LLM זה קובץ חדש ושורה אחת ב- `container.ts`".

"מה זה ה-boundaries האלה? זה לא סתם כפילות?"

"זו הפרדה בין איך שהנתונים נשמרים לאיך שהם נשלחים. הדוגמה הכי טובה היא המשתמש: ל- `UserBoundary` פשוט אין שדה סיסמה, אז היא לא יכולה לדלוף. הקוד הישן עשה `delete o.password` בשלושה מקומות — וזה דולף כברירת מחדל, כי שדה חדש בסכמה נשלח עד שמישהו יזכור למחוק אותו".

"למה SSE ולא WebSocket?"

"הזרימה חד-כיוונית — השרת מודיע ללקוח שנוסף מפגע. WebSocket דו-כיווני, כבד יותר בזיכרון, ודורש ספרייה. זה HTTP רגיל, עובד דרך כל פרוקסי, ומתחבר מחדש לבד. וזה גם מוריד עומס: במקום 450 בקשות בשעה לכל לקוח, יש חיבור אחד שקט".

"איך אתה יודע שזה עובד?"

"46 בדיקות אינטגרציה מול מסד נתונים אמיתי שעולה נקי בכל ריצה. הן מכסות את חוזה ה-JSON מול ה-Frontend, את קודי השגיאה, את ההרשאות — ויש בדיקת רגרסיה ספציפית לבאג טעינת הסביבה, שמאמת שהטוקן לא נחתם עם הסוד הישן".

אם ישאלו על חולשות — אל תתחמק

יש XSS מאוחסן ב- `MapComponent.tsx` שזוהה ומתוזמן לתיקון. ה-SSE עובד בהנחת instance יחיד; ריבוי instances היה דורש Redis pub/sub. אין עדיין pagination אמיתי — יש תקרה של 500. ו- `api/client.ts` ב-Frontend לא פוצל לשכבות כמו ב-Backend.

לומר את זה בעצמך זה סימן לבגרות הנדסית, לא לחולשה. מהנדס שמכיר את החוב שלו נשמע אחרת ממי שחושב שהכל מושלם.

שלוש נקודות שכדאי להשאיר אצל המאזין

1. באג ה- `dotenv` ב-ESM — מדגים הבנה של סדר טעינת מודולים, לא רק שימוש בספרייה.

2. `Fail-open` ב-AI — מדגים חשיבה על מצבי כשל, לא רק על המסלול המוצלח. ובמיוחד: ההבנה שכשל שקט הוא הגרוע ביותר.

3. ניקוי EXIF — מדגים מודעות לפרטיות שאיש לא ביקש.

20. מה נשאר

אבטחה — הדחוף ביותר

XSS מאוחסן ב- `MapComponent.tsx` : ה-`popup` נבנה עם `innerHTML` ומזריק לתוכו תיאור מפגע — טקסט חופשי שכל משתמש כותב, ואז כל מי שצופה במפה מריץ אותו.

זו נקודת ההזרקה **היחידה** בכל ה-`Frontend`; כל שאר המקומות מרנדרים כ-`JSX` ש-`React` בורח ממנו. והתיקון פשוט: הקומפוננטה מיותרת — יש כבר `<Popup>` ב-`JSX` שמציג את אותו תוכן.

Frontend: מעבר ל-SSE במקום פוליטציה כל 8 שניות · אירוח עצמי של אייקוני המפה (נטענים מ-GitHub ומ-cdnjs) · clustering לסמנים · פיצול `client.ts` ל- `core/` ו- `services/`

פריסה: `Dockerfile` + `docker-compose.yml` · `Nginx Proxy Manager` · `DuckDNS` · `Azure B1s` עם `proxy_buffering off` ל-SSE

מאוחר יותר: פאגינציה אמיתית · `Redis` אם אי פעם יהיה יותר מ-instance אחד